

RailOpt AI — Teammate & Developer Guide

Comprehensive Technical Architecture, Operations, and Local Engineering Handbook

Document Status: Production Reference / Active Baseline

Target Audience: Internal Software Engineers, ML/Optimization Engineers, and Railway Domain Specialists

Last Updated: 2026-09-03 (Synchronized with commit `87b0987`)

Table of Contents

- 1. [Project Overview](#)
- 2. [Problem → Solution](#)
- 3. [High-Level System Architecture](#)
- 4. [Technology Stack](#)
- 5. [Repository Structure](#)
- 6. [Application Modules & Pages](#)
- 7. [Authentication, Authorization & RBAC](#)
- 8. [Data Layer & Domain Models](#)
- 9. [Optimization Engine \(OR-Tools CP-SAT\)](#)
- 10. [AI & Explainability Architecture](#)
- 11. [Maps & Geospatial Engine](#)
- 12. [API Architecture & Endpoints](#)
- 13. [Authentication Flows: Step-by-Step](#)
- 14. [Local Development Setup](#)
- 15. [Environment Variables Reference](#)
- 16. [Production Deployment Architecture](#)
- 17. [Operational Roles & Permission Matrix](#)
- 18. [SIH Evaluation & Demo Workspace](#)
- 19. [Testing & Quality Assurance](#)
- 20. [Known Operational Considerations](#)
- 21. [Safe Change Guidelines](#)
- 22. [Quick-Start Checklist for New Teammates](#)

1. Project Overview

RailOpt AI is an enterprise-grade Decision Support System (DSS) developed for Indian Railways (benchmarked on Eastern Railway's Howrah Division). It solves multi-departmental railway maintenance coordination by transforming fragmented, manual track-possession planning into mathematically optimal, synchronized corridor possession windows.

Operational Problem

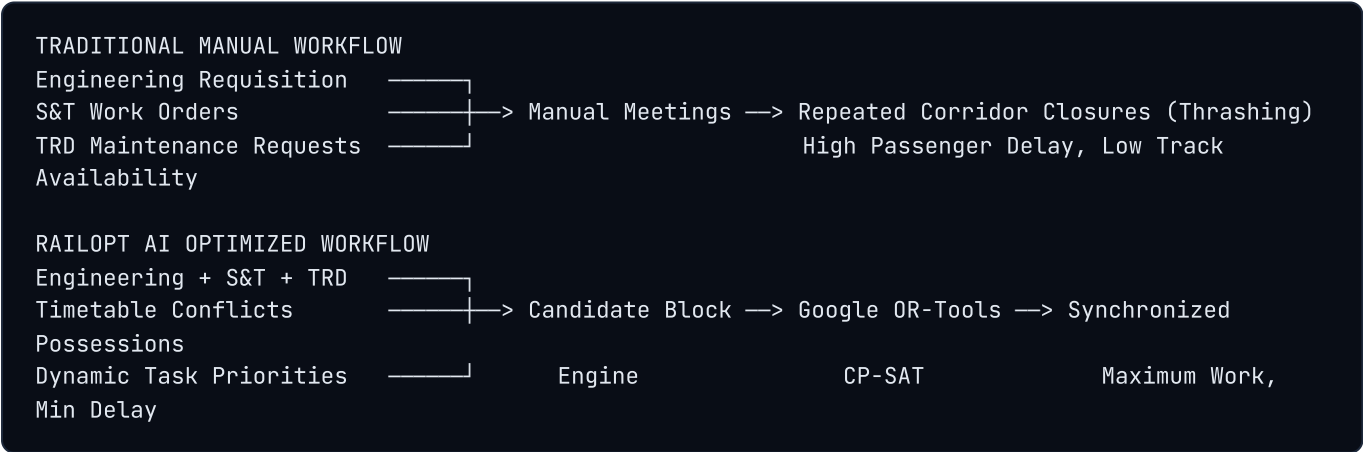
Indian Railways operates high-density mixed traffic (freight and passenger services sharing common track infrastructure). Regular physical maintenance across three independent engineering departments—**Civil Engineering (Permanent Way)**, **Signaling & Telecommunications (S&T)**, and **Traction Distribution / Overhead Equipment (TRD)**—requires track closures known as "traffic blocks." Historically planned via manual paperwork and inter-departmental negotiations, this fragmentation leads to:

- **Corridor Thrashing:** Closing the same corridor section multiple times on adjacent days for separate departments, causing repeated passenger and freight disruptions.
- **Under-Utilized Possessions:** Track blocks allocated to a single department while adjacent maintenance needs remain unaddressed.
- **Safety Risks:** Deferred urgent maintenance resulting from scheduling gridlocks.

Main Objectives & Capabilities

- **Multi-Department Consolidation:** Identify co-located cross-departmental tasks and combine them into single traffic possessions.
- **Combinatorial Window Generation:** Fast spatial-temporal generation of train-conflict-free candidate block windows across railway sections and stations.
- **Exact Optimization:** Globally solve multi-objective integer programming schedules in `< 3.0` seconds using Google OR-Tools CP-SAT.
- **Explainability:** Grounded natural-language operational summaries backed by local LLM runtimes (Ollama/Gemma 2) and cloud fallbacks, with zero hallucination risk.
- **Strict Governance:** Multi-department approval workflows (`DRAFT` `\rightarrow` `SUBMITTED` `\rightarrow` `APPROVED` / `REJECTED`) with an immutable audit trail.

2. Problem → Solution



Why Mathematical Optimization & DSS?

Human planners cannot manually evaluate tens of thousands of combinatorial combinations of track sections, safety headways, task durations, and train occupancy windows. Mathematical optimization models trade-offs explicitly: maximizing completed task priority and multi-department integration bonuses while minimizing train punctuality loss, freight delay, and unused idle window time.

3. High-Level System Architecture

RailOpt AI follows a clean decoupled microservice architecture:



Actual Request & Data Flow

- Authentication:** The user logs in via Auth0 Universal Login (or chooses a role in the SIH Demo Workspace). The browser receives a signed JWT access token.
- API Dispatch:** The Next.js frontend uses a centralized `apiClient` (`services/web/src/lib/api-client.ts`) that automatically attaches the `Authorization: Bearer` header.
- RBAC Validation:** FastAPI inspects the token signature using the Auth0 JWKS endpoint (or local secret for demo tokens), extracts `https://railopt.ai/roles` , and evaluates endpoint role requirements (`require_roles`).
- Data Retrieval & Processing:** Asynchronous SQLAlchemy 2.0 (`psycopg3`) queries the PostgreSQL/PostGIS database.
- Optimization Solve:** When a planner initiates an optimization run, `CandidateBlockEngine` generates valid time-corridor windows, and `optimizer_engine.py` runs the Google OR-Tools CP-SAT solver in-memory.
- Result Persistence & Audit:** The resulting plan and assigned blocks are committed to the database and logged to `audit_logs` .
- Explainability:** Planners requesting natural-language reasoning trigger `ExplainabilityService` , which queries Ollama (local) or Google Gemini (cloud fallback) using strictly verified context facts.

4. Technology Stack

Layer	Technology	Version	Purpose in RailOpt AI
Frontend Framework	Next.js (App Router, Turbopack)	16.3.3	Server-rendered & client-side interactive pages
UI Library & Styling	React, Tailwind CSS	19.2.8 , 4.x	Modern, responsive dark railway operational interface
Icons & Design	Lucide React	1.37.0	Enterprise operational symbology
State Management	TanStack React Query	5.102.8	Server state caching, pagination, query invalidation
Mapping & GIS	Leaflet, React-Leaflet	1.9.4	Track network geometry, station markers, possession overlays
Client Auth	oidc-client-ts , react-oidc-context	3.5.0 , 3.3.1	Standards-compliant browser OIDC PKCE flow
Backend Framework	FastAPI	0.141.1	High-performance asynchronous REST API
Python Runtime	Python (Cpython)	3.12+	Asynchronous backend services and solver logic
Optimization Solver	Google OR-Tools	9.15.6755	CP-SAT (Constraint Programming) integer solver
ORM & Database Driver	SQLAlchemy (Async), Psycopg 3	2.0.52 , 3.3.4	Async database models, queries, and connection pool
Database Migrations	Alembic	1.19.1	Version-controlled database schema migrations
Database Engine	PostgreSQL + PostGIS Extension	17+ , 3.3+	Spatial line geometries, station coordinates, relational data
Local LLM Runtime	Ollama (gemma2:2b)	CLI	Zero-cost, offline natural language plan explanation
Hosted LLM Fallback	Google Gemini API (gemini-2.5-flash)	REST	Cloud fallback when local Ollama is offline
Identity Provider	Auth0	Cloud OIDC	Centralized government-style provisioned user SSO
Testing Frameworks	Vitest, Testing Library (Web); Pytest (API)	4.1.11 (web)	Unit, integration, and end-to-end RBAC test suites

5. Repository Structure

```
railopt-ai/
├── .env.example           # Example root environment variables
├── README.md             # High-level product summary and architecture overview
├── docs/                 # Technical documentation artifacts
│   └── TEAMMATE_GUIDE.md # This comprehensive engineering guide
├── services/
│   ├── api/              # FastAPI Backend Service
│   │   ├── alembic/      # Database migration definitions
│   │   │   ├── versions/ # Migration revision scripts
│   │   │   └── env.py    # Async migration engine setup
│   │   ├── alembic.ini   # Alembic runtime configuration
│   │   ├── requirements.txt # Python dependencies
│   │   ├── seed_data.py  # Howrah Division production seed script
│   │   └── app/
│   │       ├── main.py   # FastAPI application entrypoint and middleware
│   │       ├── core/     # Core infrastructural singletons
│   │       │   ├── config.py # Pydantic BaseSettings environment manager
│   │       │   ├── database.py # SQLAlchemy async session factory & engine
│   │       │   └── security.py # JWT verification, Auth0 JWKS caching, require_roles
│   │       ├── domain/   # Pure solver domain models (contracts)
│   │       │   ├── candidate.py # OptimizationCandidate domain model
│   │       │   ├── constraints.py # HardConstraintConfig definitions
│   │       │   ├── objectives.py # ObjectiveWeights definitions
│   │       │   ├── results.py # OptimizationRunResult & SolverStatus
│   │       │   └── task.py # OptimizationTask domain model
│   │       ├── models/   # SQLAlchemy ORM database models
│   │       │   ├── admin.py # AuditLog, SystemSetting
│   │       │   ├── asset.py # Asset, MaintenanceTask
│   │       │   ├── block.py # BlockRequest, BlockRequestTask
│   │       │   ├── corridor.py # CorridorWindow, FreightForecast
│   │       │   ├── geography.py # Division, Section, Station, OperationalSubsection
│   │       │   ├── operations.py # TrainRun, TrainSectionOccupancy
│   │       │   ├── optimization.py # OptimizationRun, OptimizedBlock, OptimizedBlockTask
│   │       │   └── resource.py # Resource, ResourceAvailability
│   │       ├── routers/  # FastAPI HTTP route handlers
│   │       │   ├── assets.py # /api/v1/assets
│   │       │   ├── audit.py # /api/v1/audit
│   │       │   ├── auth.py # /api/v1/auth/demo-token
│   │       │   ├── candidate_blocks.py # /api/v1/candidate-blocks
│   │       │   ├── explanations.py # /api/v1/explanations
│   │       │   ├── health.py # /health, /health/db
│   │       │   ├── maintenance.py # /api/v1/maintenance-tasks
│   │       │   ├── optimization.py # /api/v1/optimization/runs, approvals
│   │       │   ├── scenarios.py # /api/v1/scenarios
│   │       │   ├── sections.py # /api/v1/sections
│   │       │   └── stations.py # /api/v1/stations
│   │       ├── schemas/  # Pydantic v2 validation and serialization schemas
│   │       └── services/ # Business logic engines
│   │           ├── asset_service.py
│   │           ├── candidate_block_engine.py # Time-window generation & conflict checks
│   │           ├── compatibility_engine.py # Multi-department grouping logic
│   │           ├── explainability_service.py # Advisory LLM prompt & synthesis
│   │           ├── llm_providers.py # Ollama, Gemini, Deterministic providers
│   │           └── maintenance_service.py
```

```

├── optimization_service.py    # Solve execution & persistence workflow
├── optimizer_engine.py        # Google OR-Tools CP-SAT math formulation
├── priority_engine.py          # Dynamic task priority scoring
├── web/                        # Next.js 16 Frontend Web Application
│   ├── package.json           # Node.js dependencies and run scripts
│   ├── vitest.config.ts        # Vitest test runner configuration
│   └── src/
│       ├── app/               # Next.js App Router pages
│       │   ├── layout.tsx      # Global RootLayout with Providers and AppShell
│       │   ├── page.tsx        # Root page (redirects to /dashboard)
│       │   ├── login/page.tsx  # Auth0 sign-in and SIH Demo Workspace
│       │   ├── auth/callback/page.tsx # OIDC Authorization Code exchange handler
│       │   ├── dashboard/page.tsx # Divisional operational overview & KPIs
│       │   ├── maintenance/page.tsx # Work order queue and task drawers
│       │   ├── planning/page.tsx # Integration opportunities & candidate explorer
│       │   ├── planning/calendar/page.tsx # Multi-day calendar & timeline view
│       │   ├── optimization/page.tsx # CP-SAT solver configuration & plan reviews
│       │   ├── operations/page.tsx # Live possession feed launcher
│       │   ├── map/page.tsx     # Leaflet GIS railway network map
│       │   ├── approvals/page.tsx # Official approval workflow portal
│       │   ├── audit/page.tsx   # Immutable audit log review portal
│       │   └── unauthorized/page.tsx # 403 Access Restricted landing
│       ├── components/         # Modular UI components
│       │   ├── auth/           # LogoutDialog, user badges
│       │   ├── brand/          # RailOptLogo with theme contrast support
│       │   ├── dashboard/      # KPI cards, queue previews, charts
│       │   ├── feedback/       # ForbiddenState, LoadingState, ErrorState
│       │   ├── layout/         # AppShell, Sidebar, Header, PageHeader
│       │   ├── map/            # RailwayNetworkMap, MapLegend, MapFilters
│       │   └── optimization/    # SolverLoadingState, OptimizationResultView
│       └── lib/                # Client libraries and utilities
│           ├── api-client.ts    # Typed fetch wrapper attaching auth tokens
│           ├── auth-config.ts   # OIDC config, role extraction, route gating
│           ├── providers.tsx    # React Query and OIDC Context providers
│           └── api/             # Modular API client functions

```

6. Application Modules & Pages

Every page in RailOpt AI serves a distinct operational purpose:

1. Dashboard (/dashboard):

- High-level divisional overview of track maintenance operations.
- Displays real-time KPI metrics: active work orders, overdue critical tasks, cross-department opportunities, and solver status.
- Summaries of the maintenance queue, candidate windows, and latest CP-SAT runs.

1. Maintenance (/maintenance):

- Comprehensive work order ledger across Civil Engineering, S&T, and TRD.
- Interactive filtering by department, severity (**CRITICAL**, **HIGH**, **MEDIUM**, **LOW**), section, and overdue status.
- Slide-over detail drawer displaying asset location, track structure, and priority breakdown.

1. Planning (/planning):

2. **Opportunity Explorer:** Lists multi-department integration opportunities where adjacent work orders can share track possession.

3. **Candidate Block Explorer:** Lists discrete time windows available across track sections, annotated with train-conflict and duration feasibility.

1. **Calendar Planning** (`/planning/calendar`):

2. Interactive scheduling grid with Day, Week, and Month views.

3. Visual Gantt timeline displaying start/end possession hours across corridor sections.

4. Displays unassigned tasks requiring planner intervention.

1. **Optimization Planner** (`/optimization`):

2. Interface to trigger the Google OR-Tools CP-SAT mathematical solver.

3. Planners can configure objective weights (W_{priority} , $W_{\text{integration}}$, $W_{\text{disruption}}$, etc.) and horizon limits.

4. Displays detailed solver output: objective score, corridor possessions, assigned tasks, timetable impact, and advisory AI explanations.

5. Human governance actions: Submit for approval (`SUBMITTED`), Approve (`APPROVED`), or Reject (`REJECTED`).

1. **Operations** (`/operations`):

2. Daily operational execution hub linking active track possessions with live GIS tracking.

1. **Railway Network Map** (`/map`):

2. Interactive OpenStreetMap and Leaflet GIS canvas of Howrah Division.

3. Spatial visualization of stations, track section lines, pending maintenance markers, candidate possession zones, and active blocks.

4. Layer visibility toggles and section filtering.

1. **Approvals** (`/approvals`):

2. Dedicated governance portal for Block Approvers and Divisional Authorities to review submitted plans before issuance of track notices.

1. **Audit Trail** (`/audit`):

2. Compliance logging portal displaying immutable timestamps, user identities, solver parameters, and decision records.

1. **Login & Demo Access** (`/login`):

2. **Production Route:** Redirects to Auth0 Universal Login via OIDC PKCE.

3. **SIH Demo Workspace:** 8 one-click role selectors enabling judges and reviewers to simulate any operational role instantly without entering passwords.

7. Authentication, Authorization & RBAC

RailOpt AI enforces an enterprise government-grade security model:

The 8 Operational Roles

Every user in the system belongs to one or more of 8 canonical railway roles:

1. `ADMIN` : System Administrator (unrestricted platform access).

2. `PLANNER` : Divisional Train Planning Officer (triggers solver, creates plans, submits for approval).
3. `CONTROL` : Section & Train Controller (operational monitoring, solver execution).
4. `APPROVER` : Senior Divisional Operations Manager (Sr. DOM / ADRM) with authority to approve/reject block plans.
5. `ENGINEERING` : Civil Engineering (Permanent Way / Track) maintenance engineer.
6. `SNT` : Signal & Telecommunication maintenance engineer.
7. `TRD` : Traction Distribution (OHE / Electrical) maintenance engineer.
8. `VIEWER` : Read-only guest or auditor.

Auth0 Universal Login (Production)

- **Flow:** OpenID Connect Authorization Code Flow with Proof Key for Code Exchange (PKCE) implemented via `oidc-client-ts`.
- **Public Registration:** Disabled on database connections. Accounts must be provisioned by the railway administrator.
- **Custom Claim:** An Auth0 Post Login Action injects assigned roles into the custom namespaced claim: `https://railopt.ai/roles`.
- **Tokens:** Access tokens are RS256-signed JWTs containing `aud: https://railopt-ai-api`.

FastAPI Token Verification & Deny-by-Default

- In `security.py`, incoming Bearer tokens are validated against Auth0's JWKS endpoint (`/.well-known/jwks.json`) with in-memory key caching.
- Role checks use the `require_roles(*allowed_roles)` dependency. If an authenticated user does not possess at least one of the required roles (or has zero roles assigned), the API immediately aborts with **HTTP 403 Forbidden**. Unauthenticated requests return **HTTP 401 Unauthorized**.

SIH Demo Workspace vs. Production Auth0

IMPORTANT: Do not confuse Demo JWTs with Production Auth0 tokens. - **Production SSO:** Issued by Auth0 (`https://farhanmd03.us.auth0.com/`), signed with Auth0 RS256 private keys, verified via public JWKS. - **Demo Workspace:** For SIH judge evaluations, `/api/v1/auth/demo-token` issues HS256-signed tokens using a server-side secret (`DEMO_JWT_SECRET`). In production environments, `DEMO_ACCESS_ENABLED=false` disables this endpoint entirely.

8. Data Layer & Domain Models

The database uses **PostgreSQL 17** with the **PostGIS 3.3+** spatial extension.



Major Tables and Entities

1. **divisions** : Railway divisions (e.g. **ER-HWH** Howrah Division).
2. **sections** : Railway corridor track sections between junctions. Stores spatial track coordinates as PostGIS **LineString** geometries, speed limits, and electrification type.
3. **stations** : Stations along the line. Stores 3-4 letter code (e.g. **HWH** , **BWN**), station name, and spatial **Point** location.
4. **section_station_maps** : Ordered sequence mapping stations to sections with track kilometer markers.
5. **assets** : Track infrastructure assets (turnouts, signals, overhead masts, track circuits) linked to specific sections and departments (**ENGINEERING** , **SNT** , **TRD**).
6. **maintenance_tasks** : Work orders. Stores description, department, severity, estimated duration (minutes), overdue status, and required track speed restrictions.
7. **train_runs** & **train_section_occupancies** : Timetable passenger and freight train schedules with section entry/exit timestamps used to calculate train conflicts.
8. **corridor_windows** : Available traffic window headways between scheduled trains.
9. **optimization_runs** : Master record of a CP-SAT solve execution. Stores status (**DRAFT** , **SUBMITTED** , **APPROVED** , **REJECTED**), objective score, execution runtime, weights, and planner notes.
10. **optimized_blocks** : Discrete traffic possession windows scheduled by the optimizer. Stores section, start/end timestamps, and computed impact metrics.

11. `optimized_block_tasks` : Associative junction table linking maintenance tasks into scheduled possession blocks.
12. `audit_logs` : Immutable record of all system events, status transitions, and user actions.

9. Optimization Engine (OR-Tools CP-SAT)

The optimization core is implemented in `optimizer_engine.py`. It operates purely in-memory using **Google OR-Tools CP-SAT**.

Pipeline Sequence

1. **Priority Scoring** (`priority_engine.py`): Computes priority scores ($0-100$) based on severity, safety risk, overdue duration, and track criticality.
2. **Compatibility Grouping** (`compatibility_engine.py`): Identifies work orders on the same section that can be safely executed concurrently.
3. **Candidate Generation** (`candidate_block_engine.py`): Slices corridor timetables into discrete candidate possession windows, pre-filtering train conflicts and minimum duration feasibility.
4. **CP-SAT Mathematical Solve** (`optimizer_engine.py`): Solves the integer programming model.

Mathematical Formulation

The solver creates boolean decision variables:

- $y_b \in \{0, 1\}$: Whether candidate possession block b is activated.
- $x_{i,b} \in \{0, 1\}$: Whether maintenance task i is assigned to block b .

Objective Function

$$\begin{aligned} \max \sum_{b \in \mathcal{B}} & \left(W_{\text{pri}} \cdot \text{Priority}(b) + W_{\text{int}} \cdot \text{IntegrationBonus}(b) + W_{\text{tasks}} \cdot N_{\text{tasks}}(b) \right) - \sum_{b \in \mathcal{B}} \\ & \left(W_{\text{disr}} \cdot \text{Disruption}(b) + W_{\text{frt}} \cdot \text{Freight}(b) + W_{\text{idle}} \cdot \text{Idle}(b) \right) \end{aligned}$$

Note: In CP-SAT, floating-point weights are scaled by `OBJECTIVE_SCALE = 1000` to maintain exact integer arithmetic.

Hard Operational Invariants (Protected Constraints)

1. **At-Most-Once Scheduling**: $\sum_b x_{i,b} \leq 1$ for every task i .
2. **Section Exclusivity**: No two overlapping blocks can be activated on the same physical track section: $\forall \text{forall } b_1, b_2 \text{ overlapping}, y_{b_1} + y_{b_2} \leq 1$.
3. **Activation Implication**: $x_{i,b} \leq y_b$ (tasks can only be scheduled in activated blocks).
4. **Duration Feasibility**: An activated block must provide enough time for the longest task assigned to it: $\text{Duration}(b) \geq \max_{i \in b} d_i$.
5. **Planning Horizon**: All block intervals must fall strictly within the defined horizon $[T_{\text{start}}, T_{\text{end}}]$.

10. AI & Explainability Architecture

The AI layer (`explainability_service.py`) provides **grounded natural language explanations** for optimization decisions, corridor possessions, and unassigned work orders.

Strict Operational Principles

1. **Zero Decision Authority:** The LLM never schedules, optimizes, approves, or rejects track blocks. It functions purely as an advisory translation layer.
2. **Anti-Hallucination Grounding:** Prompts are constructed using strictly verified database facts wrapped in XML tags with explicit instructions forbidding the LLM from inventing train numbers, station names, or constraints.
3. **Provider Failover Hierarchy:**
 4. **Primary:** Local **Ollama** (`gemma2:2b`) running at zero cloud token cost.
 5. **Secondary:** Hosted **Google Gemini API** (`gemini-2.5-flash`) when Ollama is unavailable.
 6. **Tertiary: Deterministic Template Provider** (100% offline rule-based synthesis that guarantees an answer even with zero AI connectivity).

11. Maps & Geospatial Engine

The map workspace (`services/web/src/app/map/page.tsx`) renders Howrah Division's spatial track network.

- **Engine:** Leaflet and React-Leaflet loaded dynamically (`ssr: false`) to avoid server-side window errors.
- **Base Map:** OpenStreetMap Carto tiles.
- **Geometries:**
 - **Sections:** Rendered as interactive polyline tracks colored by electrification/operational state. Coordinates come from `/api/v1/sections`.
 - **Stations:** Rendered as circular station markers with passenger halts and junctions. Coordinates come from `/api/v1/stations`.
 - **Maintenance Tasks:** Placed along sections with severity badges. Data comes from `/api/v1/maintenance-tasks`.
- **Possession Zones:** Highlighted track segments indicating active or candidate blocks.

12. API Architecture & Endpoints

All application routes are prefixed under `/api/v1` :

Router	Method & Endpoint	Allowed Roles	Description
Health	GET /health	Public	Service uptime and version probe
Health	GET /health/db	Public	PostgreSQL & PostGIS connection check
Auth	POST /api/v1/auth/demo-token	Public (if enabled)	Acquires evaluation JWT for chosen demo role
Sections	GET /api/v1/sections	All 8 Roles	List railway sections with line geometry
Stations	GET /api/v1/stations	All 8 Roles	List stations with geographic coordinates
Assets	GET /api/v1/assets	All 8 Roles	List track assets (turnouts, OHE, signals)
Maintenance	GET /api/v1/maintenance-tasks	All 8 Roles	List maintenance work orders with filters
Maintenance	GET /api/v1/maintenance-tasks/integration-opportunities	All 8 Roles	List cross-department combination opportunities
Candidates	GET /api/v1/candidate-blocks	All 8 Roles	List generated candidate possession windows
Optimization	POST /api/v1/optimization/runs	ADMIN , PLANNER , CONTROL	Execute CP-SAT solver and persist new plan
Optimization	GET /api/v1/optimization/runs	All 8 Roles	List historical optimization plans
Optimization	GET /api/v1/optimization/runs/{id}	All 8 Roles	Retrieve specific plan details and metrics
Optimization	GET /api/v1/optimization/runs/{id}/blocks	All 8 Roles	List scheduled possession blocks for a run
Optimization	POST /api/v1/optimization/runs/{id}/submit	ADMIN , PLANNER	Submit DRAFT plan for official approval
Optimization	POST /api/v1/optimization/runs/{id}/approve	ADMIN , APPROVER	Formally approve plan (APPROVED)
Optimization	POST /api/v1/optimization/runs/{id}/reject	ADMIN , APPROVER	Formally reject plan (REJECTED)
Scenarios	POST /api/v1/scenarios	ADMIN , PLANNER , CONTROL	Create non-destructive what-if scenario
Audit	GET /api/v1/audit/logs	ADMIN , VIEWER	Retrieve immutable system and decision audit logs
Explanations	POST /api/v1/explanations	All 8 Roles	Generate natural-language plan explanation

13. Authentication Flows: Step-by-Step

Flow A: Production Auth0 OIDC Login

1. User clicks "Continue with Auth0" on /login.
2. oidc-client-ts generates PKCE code_verifier and code_challenge.
3. Browser redirects to <https://farhanmd03.us.auth0.com/authorize>.
4. User enters provisioned railway credentials.
5. Auth0 executes Post Login Action: attaches <https://railopt.ai/roles> claim.
6. Auth0 redirects to /auth/callback?code=...
7. Frontend exchanges authorization code for RS256 Access Token & ID Token.
8. Session stored in sessionStorage; user redirected to /dashboard.
9. Every subsequent API call includes "Authorization: Bearer <access_token>".
10. FastAPI validates token via Auth0 JWKS and checks required roles.

Flow B: SIH Demo Workspace Token Flow

1. Reviewer clicks a role card (e.g. "Planner") on /login.
2. Reviewer clicks "Enter Demo Workspace as Planner".
3. Frontend POSTs {"role": "PLANNER"} to /api/v1/auth/demo-token.
4. Backend verifies DEMO_ACCESS_ENABLED=true and returns signed HS256 JWT.
5. Frontend sets up User session in oidc-client-ts store.
6. Browser navigates to /dashboard with full PLANNER capabilities.

14. Local Development Setup

Prerequisites

- Python 3.12+
- Node.js 20+ & npm
- PostgreSQL 17+ with PostGIS extension (or Docker container)
- Local Ollama runtime (optional, for offline explainability)

Step 1: Database Setup

Using local PostgreSQL or Docker:

```
# Start PostgreSQL with PostGIS
docker run --name railopt-db -e POSTGRES_DB=railopt -e POSTGRES_USER=railopt -e
POSTGRES_PASSWORD=railopt_dev_password -p 5432:5432 -d postgis/postgis:17-3.5
```

Step 2: Backend Setup

```
cd services/api

# Create virtual environment
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate # Linux/macOS

# Install dependencies
pip install -r requirements.txt

# Configure environment (.env in project root or services/api/.env)
# Set DATABASE_URL=postgresql+psycopg://railopt:railopt_dev_password@localhost:5432/railopt

# Apply Alembic database migrations
python -m alembic upgrade head

# Seed Howrah Division test data
python seed_data.py

# Start FastAPI development server
uvicorn app.main:app --reload --port 8000
```

Verify: Visit `http://localhost:8000/health` and `http://localhost:8000/health/db`.

Step 3: Frontend Setup

```
cd services/web

# Install npm dependencies
npm install

# Start Next.js development server
npm run dev
```

Verify: Open `http://localhost:3000` in your browser.

15. Environment Variables Reference

CAUTION: Never commit actual passwords, private keys, or credentials to version control. Use `.env.local` or host dashboard secret managers.

Backend (`services/api/.env` or root `.env`)

Variable	Scope	Description
<code>DATABASE_URL</code>	Server Secret	PostgreSQL connection string (<code>postgresql+psycopg://user:pass@host:5432/dbname</code>)
<code>APP_ENV</code>	Configuration	<code>development</code> or <code>production</code>
<code>OIDC_ISSUER_URL</code>	Public / Config	OIDC issuer URL (<code>https://farhanmd03.us.auth0.com/</code>)
<code>OIDC_CLIENT_ID</code>	Public / Config	Client ID (<code>ARUiG1mbMmmz0i6TK3t5wrFY8otx5pr1</code>)
<code>OIDC_AUDIENCE</code>	Public / Config	Target API audience (<code>https://railopt-ai-api</code>)
<code>OIDC_JWKS_URL</code>	Public / Config	URL for Auth0 public signing keys
<code>DEMO_ACCESS_ENABLED</code>	Configuration	Enables demo token acquisition (<code>true</code> / <code>false</code>)
<code>DEMO_JWT_SECRET</code>	Server Secret	Strong \ge 32-char secret key for signing demo JWTs
<code>LLM_PROVIDER</code>	Configuration	Provider strategy: <code>auto</code> , <code>ollama</code> , <code>gemini</code> , or <code>deterministic</code>
<code>OLLAMA_BASE_URL</code>	Internal URL	Address of local Ollama runtime (<code>http://localhost:11434</code>)
<code>GEMINI_API_KEY</code>	Server Secret	Google Gemini API key for cloud explainability fallback

Frontend (`services/web/.env.local`)

Variable	Scope	Description
<code>NEXT_PUBLIC_API_BASE_URL</code>	Client Public	URL of the backend API (<code>https://railopt-ai-36j3.onrender.com</code> or <code>http://localhost:8000</code>)
<code>NEXT_PUBLIC_OIDC_ISSUER_URL</code>	Client Public	Auth0 tenant issuer URL
<code>NEXT_PUBLIC_OIDC_CLIENT_ID</code>	Client Public	Auth0 SPA application client ID
<code>NEXT_PUBLIC_OIDC_AUDIENCE</code>	Client Public	Auth0 API audience identifier
<code>NEXT_PUBLIC_DEMO_ACCESS_ENABLED</code>	Client Public	Shows/hides SIH demo role selector on login page

16. Production Deployment Architecture

RailOpt AI runs on modern cloud infrastructure:


```

[Users / Browser]
|
+---> Vercel Edge Network
|     Next.js 16 Web Application (https://railopt-ai-five.vercel.app)
|
+---> Auth0 Cloud
|     Enterprise OIDC Tenant (farhanmd03.us.auth0.com)
|
v
[Render Cloud Service]
  Uvicorn + FastAPI Web Service (https://railopt-ai-36j3.onrender.com)
  |
  v
[Supabase Cloud Database]
  PostgreSQL 17 + PostGIS 3.3 Managed Database

```

- **Frontend:** Hosted on **Vercel** with automatic HTTPS, global CDN edge caching, and Next.js Turbopack build optimization.
- **Backend API:** Deployed as a web service on **Render**, managed with automatic Git pushes to `main`.
- **Database:** Hosted on **Supabase** with PostGIS extensions, connection pooling, and SSL encryption.
- **Identity:** Managed by **Auth0** Universal Login.

17. Operational Roles & Permission Matrix

Operational Capability	ADMIN	PLANNER	CONTROL	APPROVER	ENGINEERING	SNT	TRD	VIEWER
View Dashboard & KPIs	✓	✓	✓	✓	✓	✓	✓	✓
View Maintenance Tasks	✓	✓	✓	✓	✓	✓	✓	✓
View GIS Railway Map	✓	✓	✓	✓	✓	✓	✓	✓
View Candidate Blocks	✓	✓	✓	✓	✓	✓	✓	✓
Trigger CP-SAT Solver (/runs)	✓	✓	✓	✗	✗	✗	✗	✗
Run What-If Scenarios	✓	✓	✓	✗	✗	✗	✗	✗
Submit Plan for Approval	✓	✓	✗	✗	✗	✗	✗	✗
Approve / Reject Plan	✓	✗	✗	✓	✗	✗	✗	✗
Access Audit & Compliance	✓	✗	✗	✗	✗	✗	✗	✓

18. Demo / SIH Evaluation

The **SIH Demo Workspace** allows evaluators and hackathon judges to test the application under different railway perspectives without managing passwords or waiting for email verifications:

1. Navigate to `/login`.
2. Select any of the 8 roles in the **Demo Access** matrix (e.g. `Approver` to test governance sign-offs or `Planner` to configure and solve an optimization run).

3. Click "Enter Demo Workspace as [Role]".
4. The backend issues a time-bounded evaluation token. The interface automatically adapts its sidebar, buttons, and permissions to match that role.

19. Testing & Quality Assurance

Always run tests before committing code:

Frontend Unit & Integration Tests (Vitest)

```
cd services/web
npm test -- --run
```

Validates: 21 test files, 146+ unit tests covering UI components, role extraction, permission checking, and navigation links.

Frontend TypeScript & Production Build Check

```
cd services/web
npx tsc --noEmit
npm run build
```

Validates: Strict zero TypeScript errors and Next.js production bundle compilation.

Backend Tests (Pytest)

```
cd services/api
python -m pytest tests/
```

Validates: Endpoint security, CP-SAT solver invariants, candidate block engine output, and priority scoring.

20. Known Operational Considerations

1. **Render Free-Tier Spin-Up:**
2. The production backend on Render spins down after inactivity. Initial cold-start requests may take 30–50 seconds before returning `200 OK`. Subsequent requests run at standard low latency.
3. **Candidate Block Generation Under High Concurrency:**
4. Generating 15,000+ candidate block windows evaluates timetable cross-products. On single-core free-tier cloud instances, avoid running multiple simultaneous full-division candidate generations concurrently.
5. **Database URL Scheme:**
6. Supabase/Render supply connection strings prefixed with `postgres://` or `postgresql://`. `config.py` automatically normalizes this to `postgresql+psycopg://` to ensure async Psycopg 3 driver compatibility.

21. Safe Change Guidelines

To maintain production stability, all teammates must adhere to these engineering rules:

1. **One Optimization/Feature at a Time:**
 2. Never combine unrelated features, UI tweaks, and backend refactors into a single commit.
 3. **Golden Baseline Protection:**
 4. Verify changes against the latest working commit hash. If anything breaks, revert immediately.
 5. **Verify Zero Secret Leaks:**
 6. Run `git status` and `git diff` before committing. Never commit `.env`, credentials, or temporary debug scripts containing tokens.
 7. **Preserve RBAC Invariants:**
 8. Operational endpoints must always remain protected by `require_roles`. Never disable security checks for convenience.
 9. **Alembic Migrations:**
 10. Never modify an existing applied migration file. Always create a new revision (`alembic revision -m "description"`).
-

22. Quick-Start Checklist for New Teammates

- ☐ **Clone the repository** to your local workspace.
- ☐ **Configure `.env` files:** Copy `.env.example` to `.env` and verify database connection strings.
- ☐ **Set up Python virtual environment:** Install `requirements.txt` under `services/api`.
- ☐ **Apply migrations & seed:** Run `alembic upgrade head` followed by `python seed_data.py`.
- ☐ **Launch backend:** Start Uvicorn on port `8000` and check `http://localhost:8000/health/db`.
- ☐ **Install frontend dependencies:** Run `npm install` inside `services/web`.
- ☐ **Launch frontend:** Run `npm run dev` and open `http://localhost:3000`.
- ☐ **Test authentication:** Try logging in via the SIH Demo Workspace as `PLANNER`.
- ☐ **Run test suites:** Verify `npm test -- --run` passes cleanly.
- ☐ **Read the code:** Review `optimizer_engine.py` and `candidate_block_engine.py` to understand the optimization core.